Rakamin Academy X id/x partners

Prediction Model

ID/X Partners - Data Scientist

Presented by :
Andri Laksono

Project Portfolio

This project aims to develop a machine learning model to predict loan categories based on given financial and demographic data. By analyzing various features, the model helps in classifying loan applications, assisting financial institutions in risk assessment and decision-making.

The dataset contains multiple attributes related to loan applications, such as customer details, loan amounts, credit history, and other relevant financial indicators. The data was processed to ensure quality and consistency before training the models.

Link code here!

Project explanation video here!

Rakamin Academy

Data Understanding

- The dataset consists of 466,285 rows and 74 columns
- The total number of missing values in the dataset is 9,776,227
- The dataset has 0 duplicate rows
- The dataset contains 46 columns of type float, 22 columns of type object, 6 column of type int

```
df.describe()
```
✓ 0.4s

	loan_amnt	funded_amnt	funded_amnt_inv	int_rate	installment	annual_inc	dti	delinq_2yrs	inq_last_6mths	open_acc
count	466285.000000	466285.000000	466285.000000	466285.000000	466285.000000	4.662810e+05	466285.000000	466256.000000	466256.000000	466256.000000
mean	14317.277577	14291.801044	14222.329888	13.829236	432.061201	7.327738e+04	17.218758	0.284678	0.804745	11.187069
std	8286.509164	8274.371300	8297.637788	4.357587	243.485550	5.496357e+04	7.851121	0.797365	1.091598	4.987526
min	500.000000	500.000000	0.000000	5.420000	15.670000	1.896000e+03	0.000000	0.000000	0.000000	0.000000
25%	8000.000000	8000.000000	8000.000000	10.990000	256.690000	4.500000e+04	11.360000	0.000000	0.000000	8.000000
50%	12000.000000	12000.000000	12000.000000	13.660000	379.890000	6.300000e+04	16.870000	0.000000	0.000000	10.000000
75%	20000.000000	20000.000000	19950.000000	16.490000	566.580000	8.896000e+04	22.780000	0.000000	1.000000	14.000000
max	35000.000000	35000.000000	35000.000000	26.060000	1409.990000	7.500000e+06	39.990000	29.000000	33.000000	84.000000

```
df.shape
```
✓ 0.0s

```
(466285, 74)
```

```
total_null = df.isnull().sum()
total_null.sum()
```
✓ 0.2s

```
9776227
```

```
df.duplicated().sum()
```
✓ 1.5s

```
0
```

```
df.dtypes.value_counts()
```
✓ 0.0s

```
float64    46
object     22
int64       6
Name: count, dtype: int64
```

Data Dictionary

1. Data Understnding

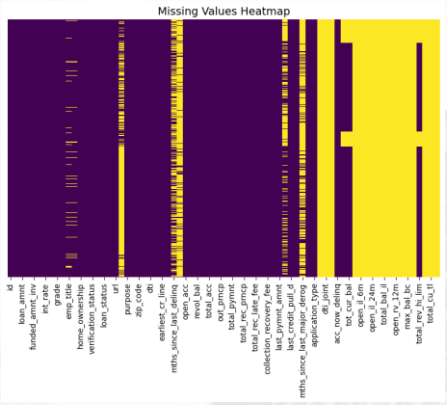
```
df.head()
```

✓ 0.0s Python

id	member_id	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	sub_grade	emp_title	emp_length	home_ownership	annu
1077501	1296599	5000	5000	4975.0	36 months	10.65	162.87	B	B2	NaN	10+ years	RENT	24
1077430	1314167	2500	2500	2500.0	60 months	15.27	59.83	C	C4	Ryder	< 1 year	RENT	30
1077175	1313524	2400	2400	2400.0	36 months	15.96	84.33	C	C5	NaN	10+ years	RENT	12

Example of DataFrame

2. Feature Engineering



This heatmap shows missing data across variables. Yellow indicates missing values, purple shows complete data. Most variables have good completeness, but several fields on the right side show significant gaps. Variables with mostly yellow have very little usable data and may need to be excluded from analysis.

```
# columns above 30% null
missing_percentage = df.isnull().sum() / len(df) * 100
columns_above_30_null = missing_percentage[missing_percentage > 30]
print(columns_above_30_null)
```
✓ 0.3s

```
desc                          72.981975
mths_since_last_delinq        53.690554
mths_since_last_record        86.566585
next_pymnt_d                  48.728567
mths_since_last_major_derog   78.773926
annual_inc_joint             100.000000
dti_joint                    100.000000
verification_status_joint    100.000000
open_acc_6m                  100.000000
open_il_6m                   100.000000
open_il_12m                  100.000000
open_il_24m                  100.000000
mths_since_rcnt_il           100.000000
total_bal_il                 100.000000
il_util                      100.000000
open_rv_12m                  100.000000
open_rv_24m                  100.000000
max_bal_bc                   100.000000
all_util                     100.000000
inq_fi                       100.000000
total_cu_tl                  100.000000
inq_last_12m                 100.000000
dtype: float64
```

```
df.drop(columns=columns_above_30_null.index, axis=1, inplace=True)
df.drop(columns=['id', 'member_id'], axis=1, inplace=True)
```
✓ 0.2s

To enhance data quality for modeling purposes, all variables exhibiting missing values exceeding the 30% threshold were systematically excluded from the feature engineering process. This methodical approach resulted in the removal of 24 features with significant data incompleteness.

The loan_status column is transformed into a binary target loan_category, where 'Current' and 'Fully Paid' are labeled as good (1) and others as bad (0). After conversion using a lambda function, the original column is dropped.

```
# Defining Target Variable (Variable Dependent)
# Kategorisasi Good (1) vs Bad (0)
good_status = ['Current', 'Fully Paid']
df['loan_category'] = df['loan_status'].apply(lambda x: 1 if x in good_status else 0)
df.drop(columns="loan_status", axis=1, inplace=True)
```
✓ 0.2s

```
# distribution percentage in column 'loan_status'
presentase_loan = df['loan_category'].value_counts(normalize=True) * 100
print(presentase_loan)
```
✓ 0.0s

```
loan_category
1    87.707089
0    12.292911
Name: proportion, dtype: float64
```

2. Feature Engineering

Data Cleaning

column 'emp_length'

```
# uniuqe value column 'emp_length'
df['emp_length'].unique()
```

```
array(['10+ years', '< 1 year', '1 year', '3 years', '8 years', '9 years',
       '4 years', '5 years', '6 years', '2 years', '7 years', nan],
      dtype=object)
```

```
# modify emp_length
df['emp_length'] = df['emp_length'].str.replace(' years', '')
df['emp_length'] = df['emp_length'].str.replace('+', '')
df['emp_length'] = df['emp_length'].str.replace('< ', '')
df['emp_length'] = df['emp_length'].str.replace('year', '')
df['emp_length'] = df['emp_length'].astype(float)
```

column 'term'

```
# unique value column 'term'
df['term'].unique()
```

```
array([' 36 months', ' 60 months'], dtype=object)
```

```
# modify column term
df['term'] = df['term'].str.replace(' months', '').astype(int)
df.rename(columns={'term': 'term_months'}, inplace=True)
```

column 'earliest_cr_line'

```
df['earliest_cr_line'].head()
```

```
0    Jan-85
1    Apr-99
2    Nov-01
3    Feb-96
4    Jan-96
Name: earliest_cr_line, dtype: object
```

```
# modify format column earliest_cr_line
df['earliest_cr_line_date'] = pd.to_datetime(df['earliest_cr_line'], format='%b-%y')
```

```
df['earliest_cr_line'].head()
```

```
0    Jan-85
1    Apr-99
2    Nov-01
3    Feb-96
4    Jan-96
Name: earliest_cr_line, dtype: object
```

```
ref_date = pd.to_datetime('2017-12-01')
df['mths_since_earliest_cr_line'] = df['earliest_cr_line_date'].apply(lambda x: (ref_date.year - x.year) * 12 + (ref_date.month - x.month))
```

+ Code + Markdown

```
df[df['mths_since_earliest_cr_line']<0][['earliest_cr_line', 'earliest_cr_line_date', 'mths_since_earliest_cr_line']].head(5)
```

	earliest_cr_line	earliest_cr_line_date	mths_since_earliest_cr_line
1580	Sep-62	2062-09-01	-537.0
1770	Sep-68	2068-09-01	-609.0
2799	Sep-64	2064-09-01	-561.0
3282	Sep-67	2067-09-01	-597.0
3359	Feb-65	2065-02-01	-566.0

```
df.loc[df['mths_since_earliest_cr_line']<0, 'mths_since_earliest_cr_line'] = df['mths_since_earliest_cr_line'].max()
```

```
ref_date = pd.to_datetime('2017-12-01')
df['mths_since_earliest_cr_line'] = df['earliest_cr_line_date'].apply(lambda x: (ref_date.year - x.year) * 12 + (ref_date.month - x.month))
```

```
df.drop(['earliest_cr_line', 'earliest_cr_line_date'], axis=1, inplace=True)
```

column 'issue_d'

```
# preprocessing issue_d
df['issue_d_date'] = pd.to_datetime(df['issue_d'], format='%b-%y')
ref_date = pd.to_datetime('2017-12-01')
df['mths_since_issue_d'] = df['issue_d_date'].apply(lambda x: (ref_date.year - x.year) * 12 + (ref_date.month - x.month))
```

```
df['mths_since_issue_d'].describe()
```

```
count    466285.000000
mean         51.255187
std          14.340154
min          36.000000
25%          41.000000
50%          47.000000
75%          57.000000
max         126.000000
Name: mths_since_issue_d, dtype: float64
```

```
df.drop(['issue_d', 'issue_d_date'], axis=1, inplace=True)
```

column 'last_pyment_d'

```
# preprocessing last_pymnt_d
df['last_pymnt_d_date'] = pd.to_datetime(df['last_pymnt_d'], format='%b-%y')
ref_date = pd.to_datetime('2017-12-01')
df['mths_since_last_pymnt_d'] = df['last_pymnt_d_date'].apply(lambda x: (ref_date.year - x.year) * 12 + (ref_date.month - x.month))
```

+ Code + Markdown

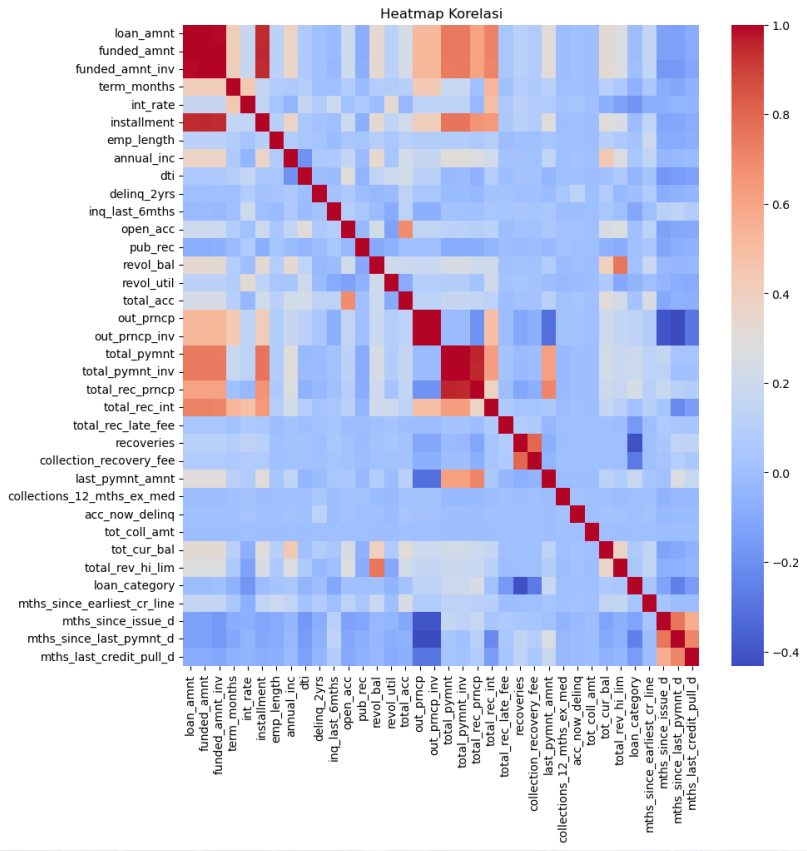
```
df['mths_since_last_pymnt_d'].describe()
```

```
count    465909.000000
mean         31.294369
std          12.805898
min          23.000000
25%          23.000000
50%          24.000000
75%          35.000000
max         120.000000
Name: mths_since_last_pymnt_d, dtype: float64
```

```
df.drop(['last_pymnt_d', 'last_pymnt_d_date'], axis=1, inplace=True)
```

Exploratory Data Analysis

Check Correlation

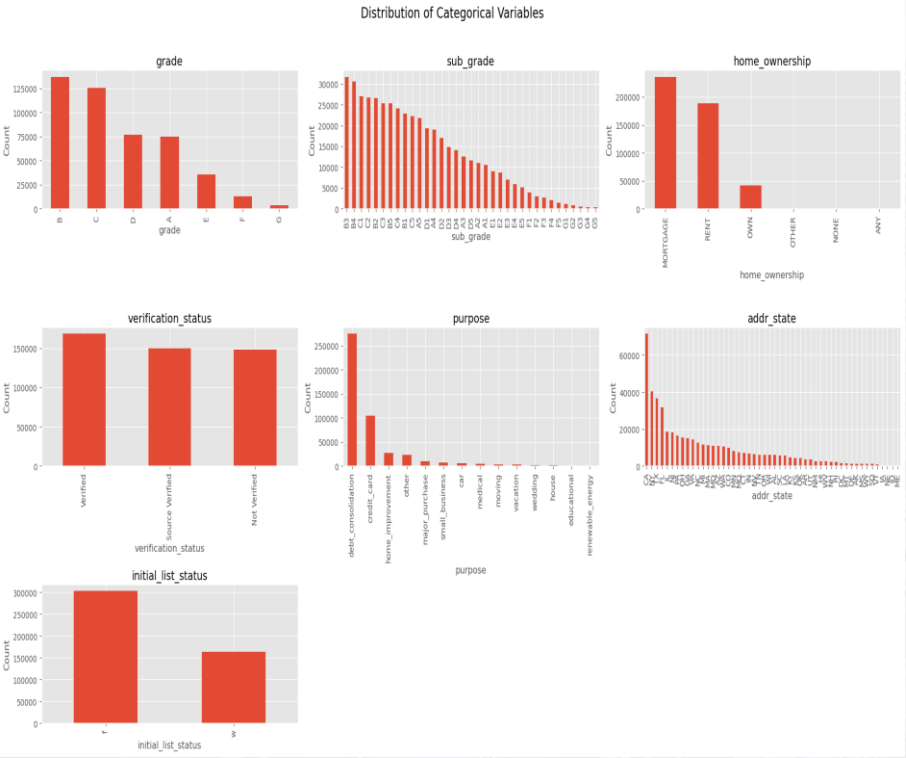


To reduce multicollinearity, highly correlated features (≥0.7) are identified, and one is removed to avoid redundancy. This improves model efficiency and interpretability

```
# Reducing Multicollinearity by Dropping Highly Correlated Features (>0.7)
corr_matrix = df_corr.abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
hicorr = [column for column in upper.columns if any(upper[column] > 0.7)]
df.drop(hicorr, axis=1, inplace=True)
```

Univariate Analysis

Categrorical Data



- **Mayoritas peminjam berada dalam grade B dan C**, sedangkan grade yang lebih rendah (F dan G) memiliki lebih sedikit peminjam, yang mungkin mencerminkan risiko lebih tinggi.

- **Sub-grade menunjukkan pola penurunan**—semakin rendah peringkatnya, semakin sedikit peminjam, yang bisa mengindikasikan bahwa lebih sedikit orang memenuhi syarat untuk kategori risiko tinggi.

- **Sebagian besar peminjam memiliki status kepemilikan rumah MORTGAGE atau RENT**, dengan sedikit yang memiliki rumah sendiri (OWN), yang dapat berpengaruh terhadap risiko kredit.

- **Status verifikasi tidak selalu lengkap**, menunjukkan bahwa masih banyak peminjam yang tidak melalui proses verifikasi penuh.

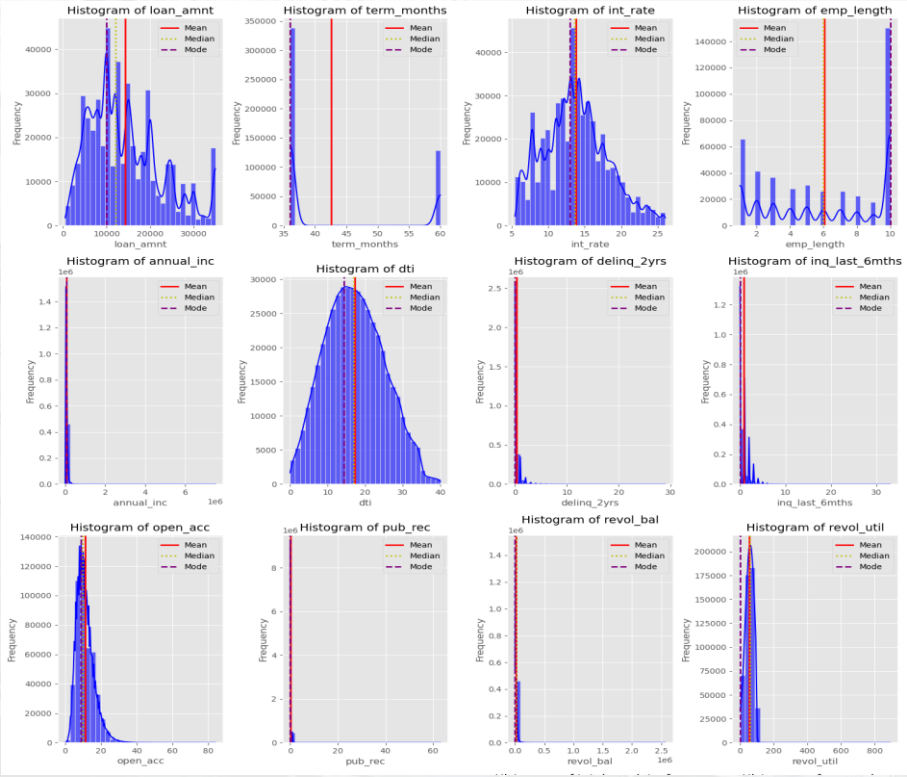
- **Tujuan utama pinjaman adalah untuk debt consolidation**, menunjukkan bahwa banyak peminjam menggunakan pinjaman untuk mengatur kembali utang mereka.

- **Distribusi peminjam tidak merata di setiap negara bagian**, beberapa wilayah memiliki lebih banyak peminjam dibandingkan yang lain.

- **Mayoritas pinjaman dimulai dalam status awal 'f'**, yang bisa mencerminkan preferensi atau aturan dalam sistem pemrosesan pinjaman.

Univariate Analysis

Numerical Data



1. Loan Amount – Sebaran jumlah pinjaman cukup bervariasi dengan beberapa puncak, menunjukkan bahwa ada beberapa nilai pinjaman yang lebih umum.

2. Term Months – Sebagian besar pinjaman memiliki jangka waktu standar (misalnya 36 atau 60 bulan).

3. Interest Rate – Distribusi tingkat bunga menunjukkan bahwa mayoritas pelanggan mendapatkan pinjaman dengan suku bunga tertentu (sekitar 10-15%).

4. Employment Length – Sebaran panjang masa kerja menunjukkan puncak pada angka tertentu, misalnya 10 tahun, yang bisa jadi karena banyaknya responden dengan pengalaman kerja lama.

5. Annual Income – Data penghasilan tahunan memiliki distribusi yang sangat miring ke kanan, menunjukkan adanya beberapa pelanggan dengan penghasilan jauh lebih tinggi dari rata-rata.

6. DTI (Debt-to-Income Ratio) – Sebagian besar pelanggan memiliki rasio utang terhadap penghasilan dalam rentang tertentu, dengan distribusi yang mendekati normal.

7. Delinquency in Last 2 Years – Mayoritas pelanggan tidak memiliki riwayat keterlambatan pembayaran dalam 2 tahun terakhir.

8. Inquiries in Last 6 Months – Sebagian besar pelanggan hanya memiliki sedikit permohonan kredit dalam 6 bulan terakhir.

9. Open Accounts – Kebanyakan pelanggan memiliki sejumlah rekening kredit terbuka dalam rentang tertentu.

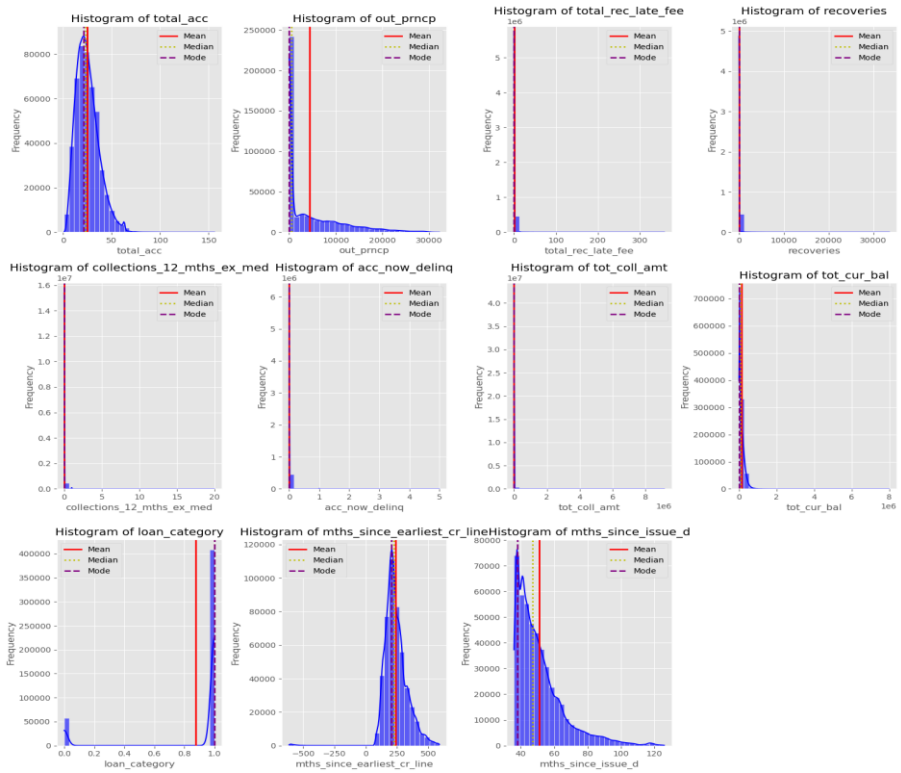
10. Public Records – Mayoritas pelanggan tidak memiliki catatan kredit publik negatif seperti kebangkrutan.

11. Revolving Balance – Sebagian besar pelanggan memiliki saldo kredit bergulir dalam kisaran tertentu.

12. Revolving Utilization – Mayoritas pelanggan menggunakan sebagian kecil dari total kredit yang tersedia.

Univariate Analysis

Numerical Data



13. Total Accounts – Sebagian besar pelanggan memiliki jumlah akun terbuka dalam jumlah yang relatif kecil, dengan distribusi yang condong ke kanan.

14. Outstanding Principal – Sebagian besar pelanggan memiliki saldo pokok yang tersisa dalam kisaran rendah, menunjukkan banyak pinjaman hampir lunas.

15. Total Received Late Fee – Mayoritas pelanggan tidak memiliki biaya keterlambatan pembayaran, ditunjukkan oleh konsentrasi tinggi di nol.

16. Recoveries – Sebagian besar pelanggan tidak memiliki pemulihan kredit atau hanya dalam jumlah kecil.

17. Collections in Last 12 Months – Hampir semua pelanggan tidak memiliki koleksi kredit dalam 12 bulan terakhir.

18. Current Delinquencies – Sebagian besar pelanggan tidak memiliki akun yang menunggak saat ini.

19. Total Collection Amount – Mayoritas pelanggan memiliki jumlah koleksi kredit yang sangat kecil atau nol.

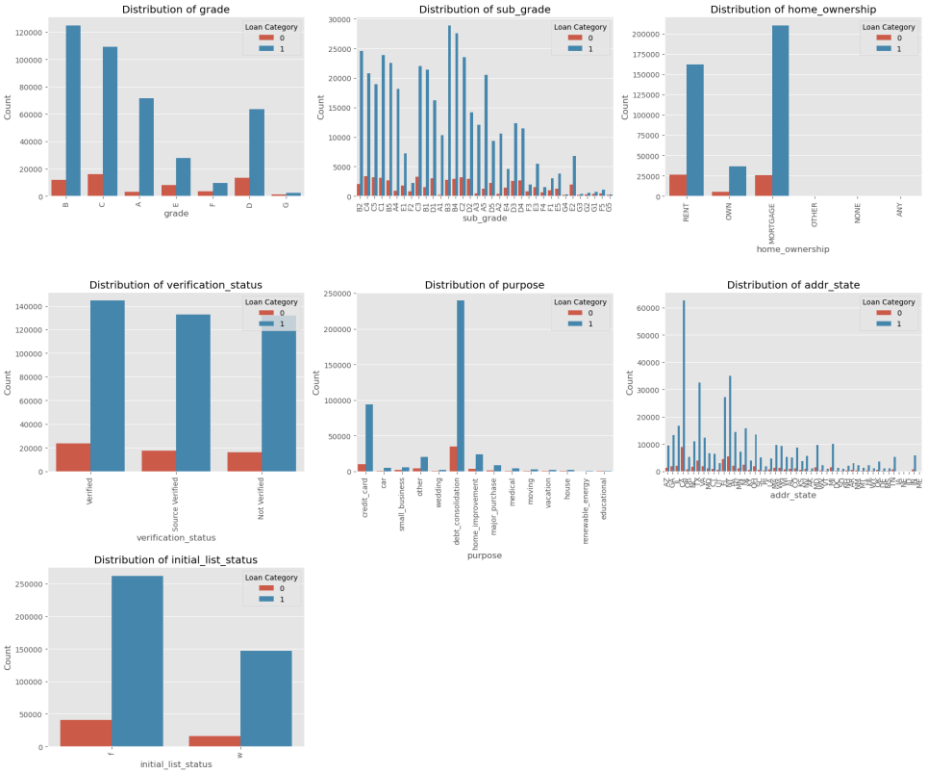
20. Total Current Balance – Sebagian besar pelanggan memiliki saldo kredit yang cukup kecil, dengan beberapa nilai ekstrem di ujung kanan.

21. Loan Category – Sebagian besar pelanggan berada dalam kategori utama pinjaman, dengan distribusi yang sangat tidak merata.

22. Months Since Earliest Credit Line – Distribusi menunjukkan bahwa mayoritas pelanggan memiliki riwayat kredit selama beberapa tahun, dengan puncak sekitar usia kredit tertentu.

23. Months Since Issue Date – Sebagian besar pinjaman dikeluarkan dalam rentang waktu tertentu, dengan distribusi yang menurun secara bertahap.

Bivariate Analysis



- **Grade & Sub_grade**: Mayoritas pinjaman di kelas B & C. Risiko gagal bayar lebih tinggi di kelas D-G.

- **Home Ownership**: Peminjam mayoritas MORTGAGE atau RENT. Pinjaman bermasalah lebih banyak di RENT.

- **Verification Status**: Sebagian besar diverifikasi, tanpa perbedaan signifikan dalam tingkat gagal bayar.

- **Purpose**: Debt consolidation paling umum. Small business lebih berisiko gagal bayar.

- **Addr State**: Pinjaman tersebar luas, tanpa pola gagal bayar spesifik per negara bagian.

- **Initial List Status**: Mayoritas ‘f’, tidak ada pengaruh besar terhadap gagal bayar

Data Preparation

Rakamin Academy

Data Imputation

```python
# Missing values filling
def fill_missing_values(df):
    for col in df.columns:
        if df[col].dtype in ['int64', 'float64']:
            df[col].fillna(df[col].median(), inplace=True)
        elif df[col].dtype == 'object':
            df[col].fillna(df[col].mode()[0], inplace=True)
    return df

df = fill_missing_values(df)
```

Standardization

```python
[ ] # standardization numerical columns
    from sklearn.preprocessing import StandardScaler

    numerical = [col for col in df.columns.tolist() if col not in categorical + ['loan_category']]
    ss = StandardScaler()
    std = pd.DataFrame(ss.fit_transform(df[numerical]), columns=numerical)
```

One Hot Encoding

```python
# One Hot Encoding categorical columns
categorical = [col for col in df.select_dtypes(include='object').columns.tolist()]
onehot = pd.get_dummies(df[categorical], dtype=int, drop_first=True)
```

Train Test Split

```python
[ ] # Train Test Split
    from sklearn.model_selection import train_test_split

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    print(f'train : {X_train.shape}')
    print(f'test : {X_test.shape}')
```

```
train : (654344, 132)
test : (163586, 132)
```

Handling Imbalance Data

```python
# Handiling Inbalance Data
from imblearn.over_sampling import SMOTE

# split features (X) and target (y)
X = df.drop(columns=['loan_category'], axis=1)
y = df['loan_category']

oversampling = SMOTE(random_state=12, sampling_strategy=1)

# fit the over sampling
X, y = oversampling.fit_resample(X, y)
```

Data Modeling

Logistic Regression

```
[ ] model = LogisticRegression()
    model.fit(X_train, y_train)
```

```
LogisticRegression()
```

```
[ ] y_pred = model.predict(X_test)
    y_pred_proba = model.predict_proba(X_test)[:, 1]
```

Random Forest Classifier

```
[ ] rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
    rf_model.fit(X_train, y_train)
```

```
RandomForestClassifier(random_state=42)
```

```
[ ] y_pred_rf = rf_model.predict(X_test)
    y_prob_rf= rf_model.predict_proba(X_test)[:, 1]
```

Gradient Boosting Classifier

```
gb_model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
gb_model.fit(X_train, y_train)
```

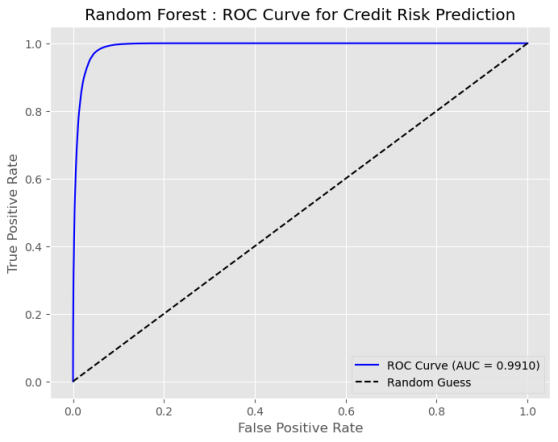
```
GradientBoostingClassifier(random_state=42)
```

```
[ ] y_pred_gb = gb_model.predict(X_test)
    y_prob_gb = gb_model.predict_proba(X_test)[:, 1]
```

Rakamin Academy

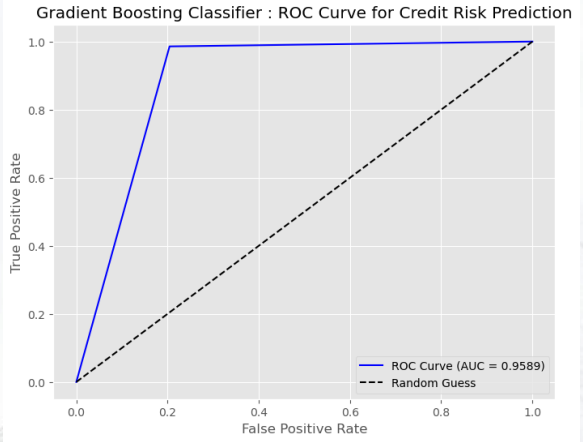
Evaluation

Random Forest Classifier



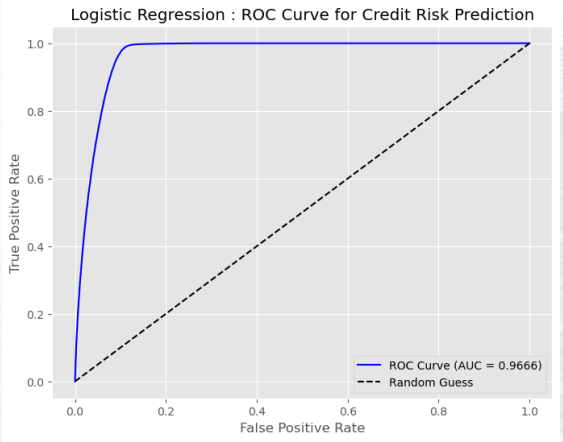

```
Random Forest
Accuracy: 0.9591
Precision: 0.9324
Recall: 0.9899
F1-score: 0.9603
ROC-AUC: 0.9910

Confusion Matrix:
 [[75956  5865]
 [  827 80938]]

Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.93      0.96     81821
           1       0.93      0.99      0.96     81765

    accuracy                           0.96    163586
   macro avg       0.96      0.96      0.96    163586
weighted avg       0.96      0.96      0.96    163586
```

Gradient Boosting Classifier




```
gradient boosting classifier
Accuracy: 0.8906
Precision: 0.8281
Recall: 0.9857
F1-score: 0.9001
ROC-AUC: 0.9589

Confusion Matrix:
 [[65094 16727]
 [ 1168 80597]]

Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.80      0.88     81821
           1       0.83      0.99      0.90     81765

    accuracy                           0.89    163586
   macro avg       0.91      0.89      0.89    163586
weighted avg       0.91      0.89      0.89    163586
```

Logistic Regression




```
Logistic Regression
Accuracy: 0.9387
Precision: 0.8978
Recall: 0.9901
F1-score: 0.9417
ROC-AUC: 0.9666

Confusion Matrix:
 [[72607  9214]
 [  808 80957]]

Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.89      0.94     81821
           1       0.90      0.99      0.94     81765

    accuracy                           0.94    163586
   macro avg       0.94      0.94      0.94    163586
weighted avg       0.94      0.94      0.94    163586
```

7. Conclusion

In this project, we implemented three machine learning models—Logistic Regression, Random Forest Classifier, and Gradient Boosting Classifier—to predict loan categories. The dataset was split into 80% training and 20% testing, ensuring a balanced evaluation of model performance.

After training and evaluating the models, we observed differences in their accuracy and efficiency. Logistic Regression provided a fast yet relatively simple approach, while Random Forest and Gradient Boosting delivered more robust predictions at the cost of longer training times.

Choosing the best model depends on the specific needs of the application. If interpretability and speed are priorities, Logistic Regression may be suitable. However, for higher predictive accuracy, ensemble methods like Random Forest and Gradient Boosting are preferred. Further optimization and feature engineering could enhance model performance.

Thank You